



Security Assessment

Nearly.Trade

Verified on October 2, 2026

SUMMARY

Project

Nearly.Trade

CHAIN

NEAR

METHODOLOGY

Manual & Automatic Analysis

FILES

Tax Token

DELIVERY

October 2, 2026

TYPE

Standard Audit



3 Total Findings 0 Critical 0 Major 0 Medium 2 Minor 1 Informational 3 Resolved

0 Critical

An exposure that can affect the contract functions in several events that can risk and disrupt the contract

0 Major

An opening & exposure to manipulate the contract in an unwanted manner

0 Medium

An opening that could affect the outcome in executing the contract in a specific situation

2 Minor

An opening but doesn't have an impact on the functionality of the contract

1 Informational

An opening that consists information but will not risk or affect the contract

3 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS

✓ **AUDIT PASSED**

TABLE OF CONTENTS | Nearly.Trade

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details

Findings

Issues
Rust Attacks
Fixes & Recommendation
Audit Comments

DISCLAIMER | Nearly.Trade

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

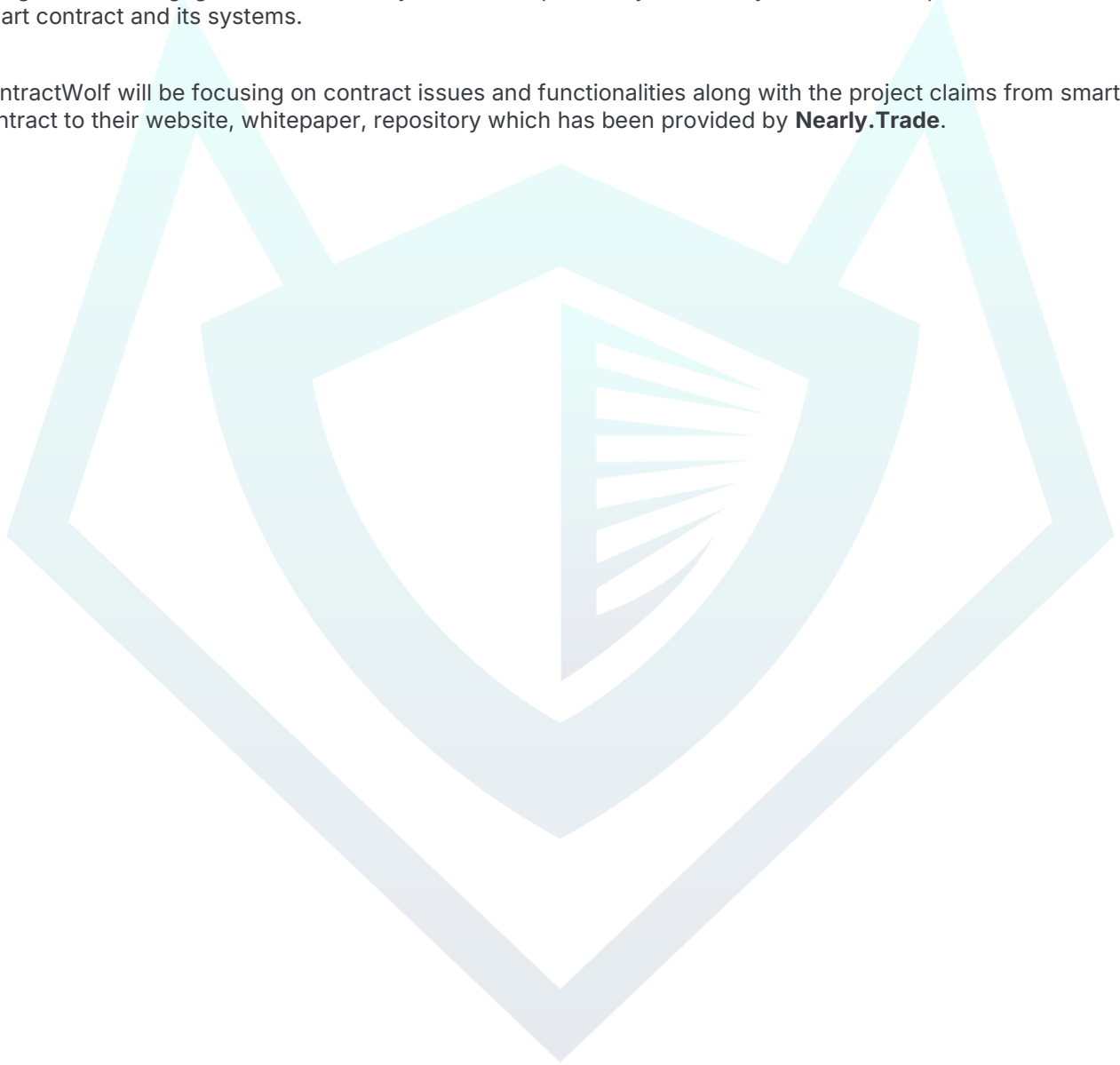
Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | Nearly.Trade

Nearly.Trade team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Nearly.Trade**.



AUDITING APPROACH | Nearly.Trade

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | Nearly.Trade



Nearly.trade is a token launchpad on NEAR. Every launch creates a token and opens a single sided liquidity position on Rhea's DCL, held by our locker contract. Creators earn 70% of pool fees. Fees on \$NEARLY are used to buy back and burn the token.

SOURCE

Source

Github Repository

Tax Token Code Hash

HS4R2isPS7hnnY8k2QbxnLHUx9Z9v9mNZnLsrGqwhChh

Raw Token Code Hash

DFiuocosGQbx3raM13ACEauZqLXXYzX77MJNf361385R

FINDINGS

Nearly.Trade



3

0

0

0

2

1

3

Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

This report has been prepared to state the issues and vulnerabilities for Nearly.Trade through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
TAX- VENUE-01	Tax Is Pinned to One Exchange Account for Good	token-tax/lib.rs	Minor	Resolved
MAXWALLE T-PAYOUT-01	Max-Wallet Rule Refuses Payouts After the Swap Ran	token-tax/lib.rs	Minor	Resolved
TAX-LP-01	Liquidity Providers Pay the Tax on Deposits, Withdrawals and Fees	token-tax/lib.rs	Informational	Resolved

RUST ATTACKS | Nearly.Trade

Rust Smart Contract Weakness Classification and Test Cases

ID	Name	Description	Status
RCW-001	Reentrancy	A malicious contract calls back into the calling contract before the first invocation of the function is finished.	● Passed
RCW-002	Undefined behavior	The Rust reference contains a non-exhaustive list of behaviors considered undefined in Rust.	● Passed
RCW-003	Integer overflow & underflow	Overflow or underflow of mathematical operations inside the Rust smart contract.	● Passed
RCW-004	Out of bounds read/write	The contract or function reads data past the end, or before the beginning, of the intended buffer.	● Passed
RCW-005	Memory corruption	Wrong usage of the memory model throughout the contract or within its functions.	● Passed
RCW-006	Typographical error	Unintended error in contract and function names, code, and arithmetic inputs.	● Passed
RCW-007	Hash collisions with multiple arguments	If used incorrectly, triggers a hash collision while calling a function within a function.	● Passed
RCW-008	Account confusion	Failure to verify account ownership or types, leading to unauthorized access.	● Passed
RCW-009	Insufficient rent exemption	Accounts not properly funded for rent exemption, risking purge.	● Passed
RCW-010	Unverified PDA derivations	Not validating Program Derived Address (PDA) seeds or bump.	● Passed
RCW-011	Missing authority checks	Unauthorized token minting via unverified authorities, and improper freeze authorities.	● Passed
RCW-012	Arithmetic precision loss	Truncation errors in financial calculations and default round-offs (e.g. 0.999 → 0).	● Passed
RCW-013	Compute unit exhaustion	Unbounded loops causing transaction failures.	● Passed
RCW-014	Reward distribution flaws	Inaccurate reward pro-rata calculations.	● Passed
RCW-015	Slot-based race conditions	Front-running via transaction ordering.	● Passed
RCW-016	Uncontrolled allocations	Memory allocation without size limits.	● Passed

FIXES & RECOMMENDATION

TAX-VENUE-01 | Tax Is Pinned to One Exchange Account for Good

contracts/token-tax/src/lib.rs

The tax applies only to transfers into or out of an account in `pairs`, fixed at new to the launch's exchange, `dclv2.ref-labs.near`, with no method to change it. A Ref v1 pool or any other venue for the token trades tax-free, and if Rhea moves accounts (`set_dcl` with `migrate`), every existing tax token loses its tax for good. `dcl_id()` is hardcoded too, so auto-registration never follows a new exchange.

```
// token-tax/lib.rs - L563 a buy or sell is a transfer out of or into `pairs`
let bps = if list_has(pairs, from) && !exempt(to) {
    buy.map(parse_u128).unwrap_or(0)
} else if list_has(pairs, to) && !exempt(from) {
    sell.map(parse_u128).unwrap_or(0)
} else {
    0
};

// token-tax/lib.rs - L589 the exchange is pinned in the code as well
fn dcl_id(me: &[u8]) -> &'static [u8] {
    if me.ends_with(b".testnet") {
        b"dclv2.ref-dev.testnet"
    } else {
        b"dclv2.ref-labs.near"
    }
}
```

Recommendation

In the next token build, let the admin append exchanges to pairs (add-only, so a tax can be extended but never lifted) and take the auto-registration source from pairs instead of a constant. Until then, document the tax as DCL-only and keep tax launches out of a set_dcl migration.

```
/// admin (the factory): add an exchange to `pairs`; add-only
#[no_mangle]
pub extern "C" fn tax_add_pair() {
    let t = sread(b"t").unwrap_or_else(|| die("no tax"));
    if find_str(&t, "admin") != Some(predecessor().as_slice()) {
        die("admin only");
    }
    let inp = input();
    let pair = find_str(&inp, "pair").unwrap_or_else(|| die("pair"));
    check_account(pair);
    swrite(b"t", &with_pair_appended(&t, pair));
}

// internal_transfer: auto-register the payouts of any listed pair
None if is_pair(from) && auto_register(to) => 0,
```

MAXWALLET-PAYOUT-01 | Max-Wallet Rule Refuses Payouts After the Swap Ran

contracts/token-tax/src/lib.rs

When `max_wallet_bps` is set, `check_max_wallet` panics on any transfer that would lift a non-exempt wallet over the cap, including Rhea's payout of a buy. By then the swap has executed and the buyer's input is spent, so the payout fails after the fact and DCL parks the tokens, as the token's own header describes for unregistered wallets. The rule is off on mainnet (`max_wallet_bps: 0`) but `set_config` can enable it.

```
// token-tax/lib.rs - L640 the cap is checked on DCL's payout, after the swap ran
let tax = tax_for(from, to, amount);
let net = amount - tax;
let nb = tb.checked_add(net).unwrap_or_else(|| die("Balance overflow"));
check_max_wallet(to, nb);

// token-tax/lib.rs - L541
if new_balance > cap {
    die("Launch window: a wallet may not hold more than the max wallet share yet");
}
```

Recommendation

Never cap a payout from a pair, since refusing it cannot undo the swap. Keep the rule off in the factory until a token build enforces it without panicking on pair payouts, for example by skipping the check when `from` is in pairs.

```
// token-tax: a pair's payout follows a swap that already ran; never refuse it
fn check_max_wallet(from: &[u8], to: &[u8], new_balance: u128) {
    let Some(rules) = sread(b"r") else { return };
    if is_pair(from) {
        return;
    }
    // ... the window check as today
}

// factory Config::validate(): keep the rule off for current token builds
require!(self.max_wallet_bps == 0, "max wallet: unsafe with DCL payouts");
```

TAX-LP-01 | Liquidity Providers Pay the Tax on Deposits, Withdrawals and Fees

contracts/token-tax/src/lib.rs

The tax is keyed on transfers, not swaps, so every movement between a holder and DCL counts as a trade: adding liquidity is taxed as a sell, and removing it or withdrawing earned fees as a buy, up to 4% each way. Third-party liquidity providers in the token's pools pay twice on capital that never traded. This is a sound trade-off, since range orders would otherwise trade tax-free, but LPs are not told.

Recommendation

Keep taxing liquidity flows (exempting them would let range orders trade tax-free), but state on the launch page and in the docs that adding liquidity, removing it and claiming fees on a tax token's pools each pay the tax.

AUDIT COMMENTS | Nearly.Trade

Smart Contract audit comment for a non-technical perspective

- The tax applies only on Rhea DCL: other exchanges trade the token tax-free, and a Rhea migration would switch the tax off for good - RESOLVED
- Providing liquidity on Rhea pays the tax when adding, removing and claiming fees - RESOLVED
- If the max-wallet rule is ever switched on, a buy over the cap is executed but not delivered - RESOLVED
- Supply is fixed at launch and minted once: the token has no mint, no owner and no pause
- Buy and sell tax is fixed at launch and can never exceed 4% a side
- Wallet-to-wallet transfers are never taxed
- A failed or partly used swap gives its tax back to the seller, in proportion to what came back
- The token account has no access keys and runs immutable shared code



CONTRACTWOLF

Blockchain Security - Smart Contract Audits